

API Engineering (FastAPI) — Project Recipe Card

Per-endpoint 5-step · layered arch · auth · DB · validation · middleware · streaming · OWASP. Deep companion: api_engineering_cheatsheet.md · Stack: FastAPI ≥0.115 · Pydantic v2 · SQLAlchemy 2.0 · Python 3.12+

◆ Per-endpoint 5-step pattern

- 1. **DECLARE** — verb, path, response_model, status, OpenAPI tags
- 2. **VALIDATE** — Pydantic request body @ boundary (422 on bad data)
- 3. **INJECT** — Annotated[T, Depends(...)] : session, settings, user
- 4. **DELEGATE** — service does the actual work (routes don't touch SQL)
- 5. **RETURN** — value conforming to response_model (filters output)

Smell test: if the route imports select from SQLAlchemy, SQL has leaked out of the repository layer.

◆ Canonical endpoint (paste-able)

```
@router.post("/trades", response_model=TradeRead,
              status_code=status.HTTP_201_CREATED)
async def create_trade(
    payload: TradeCreate,           # 2. validate
    session: SessionDep,           # 3. inject (DB)
    user:    CurrentUser,          # inject
    (auth+identity)
) -> TradeRead:
    return await TradeService(session).create(payload,
owner_id=user.id) # 4 + 5
```

◆ Project-level wiring (once at startup)

- 1. **settings** — pydantic-settings + @lru_cache; read env + .env in dev
- 2. **lifespan** — create_async_engine + sessionmaker on startup; engine.dispose() on shutdown
- 3. **middleware** — CORS, TrustedHost, GZip, security headers (order = onion; LAST added = FIRST hit)
- 4. **routers** — include versioned modules (app/api/v1/...) with prefix + tags
- 5. **handlers** — register NotFoundError→404, ConflictError→409, etc.

◆ Request flow (mental model)

```
REQUEST
→ middleware      (CORS · rate-limit · security headers ·
request-id)
→ route           (Depends graph resolved · Pydantic
validates payload)
→ service         (business logic · authz · calls repo)
→ repository      (SQL via select() + execute · returns ORM
rows)
→ service         (compose, transform)
→ route returns   → response_model filters → response_class
serialises
RESPONSE
```

Raise propagates UP until caught by an exception handler (\$10) or becomes a 500. Services raise **domain** exceptions (NotFoundError, ConflictError) — NEVER HTTPException. HTTP mapping is the routes' job.

◆ Layer responsibility (one-glance rule)

Layer	Knows	Doesn't know
routes app/api/v1/	HTTP (status, response_model, query/path/header)	SQL, business rules
services services.py	business logic, authz, transactions	HTTP, raw SQL
repositories reposi- tories.py	SQL (select() + execute)	business rules, HTTP
schemas schemas.py	I/O shapes + validators (Pydantic)	anything stateful
models models.py	ORM mapping (Mapped[] + mapped_column)	application logic

Instant smell: route imports select from SQLAlchemy → SQL leaked out of the repository.

◆ Starter · A: imports + settings

```
from contextlib import asynccontextmanager
from typing import Annotated, AsyncIterator
```

```
from functools import lru_cache
from fastapi import FastAPI, Depends, Request, status
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from sqlalchemy.ext.asyncio import AsyncSession,
create_async_engine, async_sessionmaker
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column
from sqlalchemy import select
from pydantic import BaseModel, ConfigDict, EmailStr, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict
```

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env",
extra="ignore")
    database_url: str
    secret_key: SecretStr
```

```
@lru_cache
def get_settings() -> Settings: return Settings()
SettingsDep = Annotated[Settings, Depends(get_settings)]
```

◆ Starter · B: lifespan + DB session

```
class Base(DeclarativeBase): pass
```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    engine = create_async_engine(settings.database_url,
pool_pre_ping=True)
    app.state.session_factory = async_sessionmaker(engine,
expire_on_commit=False)
    yield
    await engine.dispose()
```

```
async def get_session(req: Request) ->
AsyncIterator[AsyncSession]:
    async with req.app.state.session_factory() as s:
        yield s
SessionDep = Annotated[AsyncSession, Depends(get_session)]
```

◆ Starter · C: app + first endpoint

```
app = FastAPI(lifespan=lifespan, title="Trades API",
version="1.0.0")
# Middleware ORDER matters: last added = first executed (outer →
inner = onion)
app.add_middleware(GZipMiddleware, minimum_size=500)
app.add_middleware(TrustedHostMiddleware,
allowed_hosts=["api.example.com"])
app.add_middleware(CORSMiddleware,
allow_origins=["https://app.example.com"],
allow_methods=["GET", "POST", "PATCH", "DELETE"],
allow_headers=["Authorization", "Content-Type"])
```

```
class User(Base):
    tablename = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(unique=True, index=True)
```

```
class UserRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    email: EmailStr
```

```
@app.get("/users/{uid}", response_model=UserRead)
async def get_user(uid: int, session: SessionDep) -> UserRead:
    user = await session.get(User, uid) # PK lookup: cheaper
    than select+where
    if user is None: raise NotFoundError("User")
    return user
```

● Lifespan (startup / shutdown)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP
    engine = create_async_engine(settings.database_url,
pool_pre_ping=True,
                                pool_size=10, max_overflow=20)
    app.state.session_factory = async_sessionmaker(engine,
expire_on_commit=False)
    # warm caches, ping deps, etc.
    yield
    # SHUTDOWN
    await engine.dispose()
```

```
app = FastAPI(lifespan=lifespan)
Deprecated: @app.on_event("startup") / @app.on_event("shutdown").
Use lifespan only.
```

● Dependency injection (Annotated)

```
from typing import Annotated
from fastapi import Depends

# 1. Define type aliases ONCE (reuse everywhere)
SettingsDep = Annotated[Settings, Depends(get_settings)]
SessionDep = Annotated[AsyncSession, Depends(get_session)]
CurrentUser = Annotated[User, Depends(get_current_user)]
```

```
# 2. Use in route signatures
@router.get("/me")
async def me(user: CurrentUser) -> UserRead:
    return user
```

```
# 3. Dependencies can depend on each other (sub-deps)
async def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
    session: SessionDep,
) -> User: ...
```

Deprecated: def route(x = Depends(dep)). Use Annotated[T, Depends(dep)] — preserves typing for mypy + makes aliases reusable.

● ORM models (SQLAlchemy 2.0)

```
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column, relationship
from sqlalchemy import String, ForeignKey, DateTime, Numeric
from datetime import datetime, timezone
from decimal import Decimal
```

```
class Base(DeclarativeBase): pass
```

```
class User(Base):
    tablename = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(254), unique=True,
index=True)
    password_hash: Mapped[str] = mapped_column(String(256))
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc)) #
datetime.utcnow deprecated in 3.12+
    trades: Mapped[list["Trade"]] = relationship(
        back_populates="owner", cascade="all, delete-orphan")
```

```
class Trade(Base):
    tablename = "trades"
    id: Mapped[int] = mapped_column(primary_key=True)
    owner_id: Mapped[int] =
mapped_column(ForeignKey("users.id"), index=True)
    symbol: Mapped[str] = mapped_column(String(10), index=True)
    qty: Mapped[Decimal] = mapped_column(Numeric(18, 4))
    price: Mapped[Decimal] = mapped_column(Numeric(18, 4))
    owner: Mapped["User"] =
relationship(back_populates="trades")
```

Use Mapped[T] + mapped_column(...) (2.0 style). Legacy Column() still works but loses typing.

● Async DB sessions + 2.0 queries

Engine + session factory

```
from sqlalchemy.ext.asyncio import (create_async_engine,
async_sessionmaker, AsyncSession)
engine = create_async_engine("postgresql+asyncpg://...",
pool_pre_ping=True)
SessionFactory = async_sessionmaker(engine,
expire_on_commit=False)
```

Query patterns

from sqlalchemy import select, update, delete, func

SCALAR – single column
email = await session.scalar(select(User.email).where(User.id == uid))

SCALARS – list of ORM objects
trades = (await session.scalars(
 select(Trade).where(Trade.owner_id == uid).order_by(Trade.created_at.desc())
)).all()

COUNT
n = await
session.scalar(select(func.count()).select_from(Trade))

JOIN
stmt = select(Trade, User).join(User, Trade.owner_id == User.id)
rows = (await session.execute(stmt)).all()

UPDATE / DELETE
await session.execute(update(Trade).where(Trade.id == tid).values(price=p))
await session.commit()

Deprecated: session.query(Model).filter(...).all(). Always select() + execute/scalars/scalar.

Eager loading (avoid N+1)

```
from sqlalchemy.orm import selectinload, joinedload
stmt = select(User).options(selectinload(User.trades)) #
SELECT IN (separate query)
stmt = select(User).options(joinedload(User.trades)) # LEFT
OUTER JOIN
```

▲ Pydantic v2 — validation @ the boundary

Typed fields (use the specific type)

```
from pydantic import BaseModel, ConfigDict, EmailStr, SecretStr,
HttpUrl, Field, field_validator
from decimal import Decimal
from datetime import datetime
```

```
class TradeCreate(BaseModel):
    symbol: str = Field(..., min_length=1, max_length=10,
pattern=r"^[A-Z]+$")
    qty: Decimal = Field(..., gt=0, decimal_places=4)
    price: Decimal = Field(..., gt=0, decimal_places=4)
    notes: str | None = Field(None, max_length=500)
```

```
@field_validator("symbol")
@classmethod
def upper(cls, v: str) -> str: return v.upper()
```

```
class TradeRead(BaseModel):
    model_config = ConfigDict(from_attributes=True) # from ORM
    id: int
    symbol: str
    qty: Decimal
    price: Decimal
    created_at: datetime
```

```
class UserCreate(BaseModel):
    email: EmailStr # validated email format
    password: SecretStr # hidden in repr/logs
    site: HttpUrl | None = None
```

Model validator (cross-field rules)

```
from pydantic import model_validator
from typing_extensions import Self
```

```
class DateRange(BaseModel):
    start: datetime
    end: datetime

    @model_validator(mode="after")
    def check_range(self) -> Self:
        if self.end <= self.start:
            raise ValueError("end must be after start")
        return self
```

Computed field

```
from pydantic import computed_field
```

```
class TradeRead(BaseModel):
    qty: Decimal; price: Decimal
    @computed_field
    @property
    def gross(self) -> Decimal:
        return self.qty * self.price
```

▲ Routes — status, response model, OpenAPI

```
from fastapi import APIRouter, status, Query, Path, Header
router = APIRouter(prefix="/v1/trades", tags=["trades"])
```

```
@router.get(
   ("/{trade_id}",
    response_model=TradeRead,
    responses={404: {"description": "Trade not found"}},
    summary="Fetch a trade by ID",
)
async def get_trade(
    trade_id: Annotated[int, Path(gt=0)],
    session: SessionDep,
    user: CurrentUser,
    x_request_id: Annotated[str | None, Header()] = None,
) -> TradeRead:
    return await TradeService(session).get(trade_id, user.id)
```

```
@router.get("/", response_model=list[TradeRead])
async def list_trades(
    session: SessionDep, user: CurrentUser,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    cursor: Annotated[int | None, Query()] = None,
): ...
```

Verb	Default status	Use
GET	200	read; idempotent
POST	201	create resource
PATCH	200	partial update
PUT	200/204	full replace (idempotent)
DELETE	204	no content on success

◆ Auth — JWT + Argon2 (canonical 2026)

Password hashing (pwdlib + Argon2id)

```
from pwdlib import PasswordHash
from pwdlib.hashers.argon2 import Argon2Hasher
password_hash = PasswordHash((Argon2Hasher(),))
```

```
def hash_pw(plain: str) -> str: return password_hash.hash(plain)
def verify_pw(plain: str, hashed: str) -> bool:
    return password_hash.verify(plain, hashed)
```

Argon2id (memory-hard) is OWASP's current first choice. Bcrypt OK as a fallback.

JWT issuance + decode (PyJWT — canonical 2026)

```
import jwt # uv add
pyjwt[crypto]
from datetime import datetime, timedelta, timezone
import uuid
```

```
ALG = "HS256"
ISS = "api.example.com"
AUD = "api.example.com"
```

```
def create_token(sub: str, secret: str, minutes: int = 15) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": sub,
        "iat": now,
        "exp": now + timedelta(minutes=minutes),
        "iss": ISS, "aud": AUD, # prevent
cross-service replay
        "jti": str(uuid.uuid4()), # token id
(for revocation list)
    }
    return jwt.encode(payload, secret, algorithm=ALG)
```

```
def decode_token(tok: str, secret: str) -> dict:
    try:
        return jwt.decode(tok, secret, algorithms=[ALG], #
explicit list defends alg=none
                        audience=AUD, issuer=ISS,
                        options={"require": ["exp", "iat",
"iss", "aud", "sub"]})
    except jwt.PyJWTError: raise CredentialsError

Don't use python-jose — unmaintained since 2024 with unpatched CVEs
(CVE-2024-33663 algorithm-confusion, CVE-2024-33664 DoS). Use PyJWT (or
authlib).
```

Refresh-token rotation (prod auth)

```
# Short-lived ACCESS (15m) + long-lived REFRESH (7d) with
rotation + reuse detection.
@router.post("/token/refresh")
async def refresh(refresh_token: str, session: SessionDep,
settings: SettingsDep):
    payload = decode_token(refresh_token,
settings.secret_key.get_secret_value())
    jti = payload["jti"]
    if await is_revoked(jti): # reuse
detection
        await revoke_user_all(payload["sub"]) # SECURITY:
rotate everything
        raise CredentialsError
        await revoke(jti) # one-time
use -> blacklist old jti
    return {
        "access_token": create_token(payload["sub"], ...,
minutes=15),
        "refresh_token": create_token(payload["sub"], ...,
minutes=60*24*7),
        "token_type": "bearer",
    }
```

OAuth2 dep (`get\_current\_user`)

```
from fastapi.security import OAuth2PasswordBearer,
OAuth2PasswordRequestForm
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/v1/auth/token")
```

```
async def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
    session: SessionDep,
    settings: SettingsDep,
) -> User:
    payload = decode_token(token,
settings.secret_key.get_secret_value())
    user = await session.scalar(select(User).where(User.id ==
int(payload["sub"])))
    if user is None: raise CredentialsError
    return user
```

```
CurrentUser = Annotated[User, Depends(get_current_user)]
```

Login route (OAuth2 form)

```
@router.post("/token")
async def login(
    form: Annotated[OAuth2PasswordRequestForm, Depends()],
    session: SessionDep, settings: SettingsDep,
):
    user = await session.scalar(select(User).where(User.email ==
form.username))
```

```

    if user is None or not verify_pw(form.password,
user.password_hash):
    raise HTTPException(status_code=401, detail="Bad
credentials",
                        headers={"WWW-Authenticate":
"Bearer"})
    secret = settings.secret_key.get_secret_value()
    return {"access_token": create_token(str(user.id), secret,
minutes=15),
"refresh_token": create_token(str(user.id), secret,
minutes=60*24*7),
"token_type": "bearer"}

```

◆ Authorization — close BOLA (OWASP API1)

◆ Every read/write of an object MUST verify owner_id (or role) in the SERVICE.

```

class TradeService:
    def __init__(self, session: AsyncSession): self.s = session

    async def get(self, trade_id: int, user_id: int) -> Trade:
        t = await self.s.scalar(select(Trade).where(Trade.id ==
trade_id))
        if t is None: raise NotFoundError("Trade")
        if t.owner_id != user_id:
            # ← BOLA
            check
            raise ForbiddenError("Not your trade")
        return t

```

BOLA = Broken Object-Level Authz = #1 API risk (OWASP). Default-deny. Every GET / PATCH / DELETE that takes an ID must verify ownership before returning. ID-only checks (request path parses to int) are NOT enough.

◆ Error handling — one mapping

```

# app/exceptions.py — domain exceptions
class NotFoundError(Exception): pass
class ConflictError(Exception): pass
class ForbiddenError(Exception): pass
class CredentialsError(Exception): pass

```

```

# app/main.py — single registry, every domain exception → HTTP
code
from fastapi import Request
from fastapi.responses import JSONResponse

```

```

@app.exception_handler(NotFoundError)
async def _nf(req: Request, exc: NotFoundError):
    return JSONResponse(status_code=404, content={"detail":
str(exc) or "Not found"})

```

```

@app.exception_handler(ConflictError)
async def _conf(req: Request, exc: ConflictError):
    return JSONResponse(status_code=409, content={"detail":
str(exc) or "Conflict"})

```

```

@app.exception_handler(ForbiddenError)
async def _fb(req: Request, exc: ForbiddenError):
    return JSONResponse(status_code=403, content={"detail":
"Forbidden"})

```

```

# Pydantic ValidationError handled automatically by FastAPI →
422

```

NEVER include stack traces / SQL / internal error text in JSON. Log it (with request_id), return a generic message.

◆ Config — pydantic-settings + @lru_cache

```

from pydantic import SecretStr, PostgresDsn
from pydantic_settings import BaseSettings, SettingsConfigDict
from functools import lru_cache

```

```

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env",
extra="ignore",
                                case_sensitive=False)
    # nested: ENV var "DB_POOL_SIZE" overrides db.pool_size
    database_url: PostgresDsn
    secret_key: SecretStr # hidden in repr/
logs

```

```

cors_origins: list[str] = []
rate_limit_per_minute: int = 100

```

```

@lru_cache
def get_settings() -> Settings: return Settings()
Read once at startup; cached forever. Override via env vars (12-factor).
Use SecretStr for any credential — never plain str.

```

◆ Middleware stack (order = onion)

```

# LAST added = FIRST to hit the request (and LAST to see the
response).
# Add OUTER → INNER:
app.add_middleware(GZipMiddleware, minimum_size=500)
app.add_middleware(TrustedHostMiddleware,
allowed_hosts=["api.example.com"])
app.add_middleware(CORSMiddleware,
allow_origins=settings.cors_origins, # specific URLs
only
allow_credentials=True,
allow_methods=["GET", "POST", "PATCH", "DELETE"],
allow_headers=["Authorization", "Content-Type"])

# Custom security headers + request-id middleware:
@app.middleware("http")
async def sec_headers(request: Request, call_next):
    rid = request.headers.get("X-Request-Id") or
str(uuid.uuid4())
    request.state.request_id = rid
    resp = await call_next(request)
    resp.headers["X-Request-Id"] = rid
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["X-Frame-Options"] = "DENY"
    resp.headers["Referrer-Policy"] = "strict-origin-when-cross-
origin"
    resp.headers["Strict-Transport-Security"] = "max-
age=63072000; includeSubDomains"
    return resp

```

CORS: NEVER allow_origins=["*"] with allow_credentials=True — silently dropped by browsers. Use explicit origins.

◆ Pagination + rate limiting (OWASP API4)

Cursor pagination (canonical for large tables)

```

@router.get("/trades", response_model=Page[TradeRead])
async def list_trades(
    session: SessionDep, user: CurrentUser,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    cursor: Annotated[int | None, Query()] = None,
):
    stmt = (select(Trade).where(Trade.owner_id == user.id)
.order_by(Trade.id.desc()).limit(limit + 1))
    if cursor: stmt = stmt.where(Trade.id < cursor)
    rows = (await session.scalars(stmt)).all()
    next_cursor = rows[limit].id if len(rows) > limit else None
    return Page(items=rows[:limit], next_cursor=next_cursor)

```

Cursor is stable under inserts (offset isn't). Always cap limit with le=.

Rate limit (slowapi)

```

from slowapi import Limiter
from slowapi.util import get_remote_address
limiter = Limiter(key_func=get_remote_address,
default_limits=["100/minute"])
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded,
_rate_limit_handler)

```

```

@router.post("/trades")
@limiter.limit("10/minute") # per-route override
async def create_trade(request: Request, ...): ...

```

◆ Background tasks (fire-and-forget)

```

from fastapi import BackgroundTasks

@router.post("/trades")
async def create_trade(payload: TradeCreate, session:
SessionDep,
                                bg: BackgroundTasks, user: CurrentUser):
    trade = await TradeService(session).create(payload, user.id)

```

```

bg.add_task(send_confirmation_email, user.email, trade.id)
# runs AFTER response
return trade

```

Single-worker only. If the worker crashes, the task is LOST. For multi-process / durable jobs use Celery / RQ / Arq / Dramatiq.

◆ Streaming / SSE / WebSockets

NDJSON streaming (large CSV / generators)

```

from fastapi.responses import StreamingResponse
import json

@router.get("/trades/export")
async def export(session: SessionDep, user: CurrentUser):
    async def rows():
        async for t in session.stream_scalars(
            select(Trade).where(Trade.owner_id == user.id)):
            yield
    json.dumps(TradeRead.model_validate(t).model_dump()) + "\n"
    return StreamingResponse(rows(), media_type="application/x-
ndjson")

```

SSE (server-sent events) — sse-starlette

```

from sse_starlette import EventSourceResponse # uv add
sse-starlette

```

```

@router.get("/stream")
async def stream(user: CurrentUser):
    async def gen():
        async for chunk in llm.astream("..."):
            yield {"event": "message", "data": chunk.content}
    return EventSourceResponse(gen())

```

Use sse-starlette for correct SSE framing (heartbeats, retries). Don't roll your own data: ... \n \n string.

WebSockets (bidirectional)

```

from fastapi import WebSocket, WebSocketDisconnect

```

```

@app.websocket("/ws")
async def ws(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            msg = await websocket.receive_json()
            await websocket.send_json({"echo": msg})
        except WebSocketDisconnect:
            pass

```

Auth BEFORE accept(). Depends IS supported on WS routes, but you cannot raise HTTPException — reject with await websocket.close(code=1008) instead.

◆ Observability

```

import structlog, time, uuid
log = structlog.get_logger()

@app.middleware("http")
async def access_log(request: Request, call_next):
    t0 = time.perf_counter()
    rid = getattr(request.state, "request_id",
str(uuid.uuid4()))
    try:
        resp = await call_next(request)
        log.info("http", request_id=rid, method=request.method,
path=request.url.path, status=resp.status_code,
latency_ms=round((time.perf_counter()-t0)*1000,
2))
        return resp
    except Exception:
        log.exception("http_error", request_id=rid)
        raise

```

Stack: structlog (JSON logs) + opentelemetry-instrumentation-fastapi (traces) + prometheus-fastapi-instrumentator (metrics). Pin X-Request-Id on every log line.

◆ Testing — pytest + httpx + transactional rollback

```

import pytest, pytest_asyncio
from httpx import AsyncClient, ASGITransport
from sqlalchemy.ext.asyncio import create_async_engine,
async_sessionmaker

```



```
@pytest_asyncio.fixture
async def session():
    engine = create_async_engine("sqlite+aiosqlite:///memory:")
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    Sess = async_sessionmaker(engine, expire_on_commit=False)
    async with Sess() as s:
        yield s
    await engine.dispose()
```

```
@pytest_asyncio.fixture
async def client(session):
    async def override(): yield session
    app.dependency_overrides[get_session] = override
    async with AsyncClient(transport=ASGITransport(app=app),
                           base_url="http://t") as c:
        yield c
    app.dependency_overrides.clear()
```

```
@pytest.mark.asyncio
async def test_create_trade(client, auth_headers):
    r = await client.post("/v1/trades",
                          json={"symbol": "AAPL", "qty":
                                1, "price": 100},
                          headers=auth_headers)
    assert r.status_code == 201
    assert r.json()["symbol"] == "AAPL"
```

Canonical patterns: AsyncClient(transport=ASGITransport(app=app)) (no live server); dependency_overrides for swap; one engine per test for isolation.

◆ **Deploy — uvicorn + gunicorn**

```
# Production process: gunicorn manages N uvicorn workers
uv run gunicorn app.main:app \
    --workers 4 \
    # N = 2 × CPU + 1
(sync) or CPU (async)
--worker-class uvicorn.workers.UvicornWorker \
--bind 0.0.0.0:8000 \
--timeout 30 \
--graceful-timeout 30 \
--keep-alive 5 \
--access-logfile - --error-logfile -
```

```
# Docker (slim multi-stage):
FROM python:3.12-slim AS base
RUN pip install uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY . .
EXPOSE 8000
CMD ["uv", "run", "gunicorn", "app.main:app",
    "--workers", "4", "--worker-
class", "uvicorn.workers.UvicornWorker",
    "--bind", "0.0.0.0:8000"]
```

- Run behind a reverse proxy (nginx / Traefik) for TLS termination + buffering.
- HSTS + HTTPSRedirectMiddleware at the proxy side.
- **Two healthchecks:** /healthz (liveness, cheap — process alive) vs /readyz (readiness — does DB ping). Conflating them causes restart storms on transient DB blips.
- **Workers:** 2×CPU+1 for sync; for ASYNC use CPU count (over-subscribing wastes connections).
- **Migrations:** separate step (NOT at lifespan), run once per deploy before workers start.

◆ **Migrations — Alembic (async-aware)**

```
# 1. Bootstrap (one-time):
uv add alembic
alembic init -t async migrations

# 2. Edit migrations/env.py — point to your Base.metadata:
from app.models import Base
target_metadata = Base.metadata
# (and read DATABASE_URL from your Settings, not alembic.ini)

# 3. Autogenerate + review + apply (per change):
alembic revision --autogenerate -m "add trades table"
# REVIEW generated SQL — autogenerate misses: index drops, enum changes,
# server default changes, custom types. Hand-edit before applying.
alembic upgrade head

# 4. Deploy ordering — migrations BEFORE new workers boot:
# a. apply alembic upgrade head (separate CI job / init container)
# b. roll the new app workers
# c. roll the old workers out

# Downgrade for rollback:
alembic downgrade -1
```

Test fixtures use Base.metadata.create_all — that's DEV ONLY. Production tables MUST come through Alembic so every schema change has a reviewed migration / rollback path.

▲ **OWASP API Top 10 (2023) — coverage**

#	Risk	Defence
API1	BOLA	owner check in service for every object access
API2	Broken Auth	JWT + Argon2 + token expiry + rotate secrets
API3	Excessive data exposure	response_model filters output to declared fields
API4	Unrestricted resource	limit caps + rate limits
API5	Broken Function Authz	role check in dep (require_role("admin"))
API6	Unrestricted business flows	idempotency keys (Idempotency-Key header → dedupe table); per-resource quotas
API7	SSRF	validate HttpUrl; block private IP ranges
API8	Security misconfig	TrustedHost · CORS specific · security headers · no debug
API9	Inventory mgmt	version routers (/v1); deprecate old versions
API10	Unsafe API consumption	validate ALL upstream API responses with Pydantic too

▲ **Async do/don't**

DO	DON'T
async def + await for all I/O	mix sync DB driver in async path
asyncio.gather(*tasks) for concurrency	blocking requests.get — use httpx.AsyncClient
asyncio.to_thread(fn) for CPU/blocking libs	blocking sleep — use asyncio.sleep
async with session / client	forget to await a coroutine (silent bug)
async generators for streaming	open and not close async resources

▲ **Top anti-patterns (6-month memory)**

- **SQL in route** → bypass service; can't unit-test business logic.
- **HTTPException in service** → couples business code to HTTP. Use domain exceptions.
- **response_model missing** → output leaks password_hash, internal IDs.
- **Sync DB driver** (psycopg2) in async path → entire worker blocks. Use asyncpg.

- **on_event("startup")** deprecated → use lifespan.
- **x = Depends(...)** deprecated → use Annotated[T, Depends(...)].
- **session.query()** legacy → use select() + execute/scalars.
- **Column()** legacy → use Mapped[] + mapped_column.
- **Plain str for password/email/URL** → use SecretStr / EmailStr / HttpUrl.
- **Stack traces in JSON errors** → leaks internals. Log it, return generic.
- **BackgroundTasks for "important" jobs** → lost on worker crash. Celery / RQ instead.
- **CORSMiddleware allow_origins=["*"]** with credentials → silently dropped.
- **Forgetting BOLA check** → user A reads user B's data. Default-deny in service.
- **Forgetting .commit()** in async session → writes silently lost. expire_on_commit=False in factory.
- **N+1 query** → use selectinload / joinedload on relationships.
- **python-jose** unmaintained + CVEs → use PyJWT.
- **datetime.utcnow** deprecated 3.12+ → datetime.now(timezone.utc).
- **slowapi without request: Request** in route signature → silent no-op. Required as first non-self param.
- **asyncpg + PgBouncer transaction pooling** conflicts (prepared statements). Use session pooling, or switch to psycopg[binary, pool] v3.
- **cors_origins=[] env parsing:** pydantic-settings expects JSON (CORS_ORIGINS='["a","b"]'), not comma-sep. Add a @field_validator if you want commas.
- **One /health** doing DB ping → restart storms on transient blips. Split /healthz (liveness) vs /readyz (readiness).

◆ **Quick-reference (recall in 5 seconds)**

Task	Canonical pattern
Startup/shutdown	@asynccontextmanager lifespan(app) + FastAPI(lifespan=lifespan)
DI	Annotated[T, Depends(dep)] + type-alias SessionDep
Settings	pydantic-settings BaseSettings + @lru_cache get_settings()
ORM model	class X(Base): id: Mapped[int] = mapped_column(primary_key=True)
Async DB	create_async_engine + async_sessionmaker(expire_on_commit=False)
Query	await session.scalars(select(M).where(...))
Eager load	.options(selectinload(M.rel))
Request body	payload: TradeCreate(Pydantic v2 BaseModel)
Response shape	response_model=TradeRead + ConfigDict(from_attributes=True)
Status	status_code=status.HTTP_201_CREATED
Password	pwdlib + Argon2Hasher
JWT	python-jose · HS256 · exp + iat always
Auth dep	OAuth2PasswordBearer(tokenUrl="/v1/auth/token")
BOLA check	service compares obj.owner_id == user.id
Errors	domain exceptions + @app.exception_handler registry
Rate limit	slowapi + @limiter.limit("10/minute")
Pagination	cursor-based (id < cursor) + limit=Query(ge=1, le=100)
Background	BackgroundTasks for fire-and-forget; Celery for durable
Streaming	StreamingResponse(async_gen(), media_type=...)
SSE	sse-starlette EventSourceResponse
WebSocket	@app.websocket("/ws") + auth BEFORE accept()
Tests	AsyncClient(transport=ASGITransport(app=app)) + dependency_overrides
Deploy	gunicorn -k uvicorn.workers.UvicornWorker -w 4
Migrations	alembic upgrade head — separate from lifespan